



CLOSEST PAIRS SUM IN TWO SORTED ARRAYS

Jean Henrique



CONTEXTO

Dados dois arrays $a[]$ e $b[]$ e um número inteiro positivo x , encontre um par $(a[i], b[j])$ tal que a soma de $a[i]$ e $b[j]$ seja a mais próxima de x . Em outras palavras, a diferença absoluta $|a[i] + b[j] - x|$ seja o menor possível.

SOLUÇÃO NAIVE

A solução mais direta é simplesmente percorrer o vetor A, e para cada A[i] percorrer todo o vetor B, somando A[i] com B[j] e verificando se é o valor mais próximo de X ou não. Se for, guarda o valor em uma variável, se não for, segue percorrendo. No entanto, para garantir achou o valor mais próximo, é obrigatório percorrer todo o vetor A, além de percorrer todo o vetor B N vezes, sendo N o tamanho do vetor A.

Complexidade $O(N \times M)$

```
def closestPair(a, b, x):  
    minDiff = sys.maxsize  
    res = (0, 0)  
  
    # Percorre todos os pares possíveis  
    for i in range(len(a)):  
        for j in range(len(b)):  
            diff = abs(a[i] + b[j] - x)  
  
            if diff < minDiff:  
                minDiff = diff  
                res = (a[i], b[j])  
  
    return res
```



NO ENTANTO...

Considerando que os 2 vetores já estão ordenados, existe uma outra maneira muito mais eficiente de encontrar a solução.



SOLUÇÃO 2

- Define-se 2 “ponteiros”: um no início do vetor A e outro no fim do vetor B.
- Calculamos a soma dos elementos apontados e sua diferença para x. Se for menor que a melhor encontrada, atualizamos o resultado.
- Se a soma for maior que x, diminuimos o ponteiro de B; se for menor, aumentamos o ponteiro de A. Isso é possível porque os vetores estão ordenados.
- Dessa forma, cada ponteiro percorre seu vetor no máximo uma vez.

Complexidade $O(N + M)$

```
def findClosestPair(a, b, x):  
    l, r = 0, len(b) - 1  
    diff = float('inf')  
    result = [0, 0]  
  
    while l < len(a) and r >= 0:  
        s = a[l] + b[r]  
        currDiff = abs(s - x)  
        if currDiff < diff:  
            diff = currDiff  
            result[0] = a[l]  
            result[1] = b[r]  
        if s > x:  
            r -= 1  
        else:  
            l += 1  
  
    return result
```

RESULTADO

$N = 1000 \times M = 1000$

Valor procurado: 1997

=== Busca Linear ===

Resultado: (997, 1000)

Soma: 1997

Tempo: 0.164517 segundos

=== Dois Ponteiros ===

Resultado: [997, 1000]

Soma: 1997

Tempo: 0.000533 segundos

=== Comparação ===

Diferença de tempo: 0.163984 segundos

Busca linear foi aproximadamente 308.72x mais lenta

$N = 10000 \times M = 10000$

Valor procurado: 19997

=== Busca Linear ===

Resultado: (9997, 10000)

Soma: 19997

Tempo: 14.031352 segundos

=== Dois Ponteiros ===

Resultado: [9997, 10000]

Soma: 19997

Tempo: 0.002596 segundos

=== Comparação ===

Diferença de tempo: 14.028756 segundos

Busca linear foi aproximadamente 5405.82x mais lenta

CONCLUSÃO

Ao aumentar o tamanho dos vetores de 1.000 para 10.000 elementos, a busca linear passou de 0.16s para 14.03s, enquanto o algoritmo de dois ponteiros passou de 0.00053s para apenas 0.00260s. Com isso, a busca linear, que possui complexidade $O(N \times M)$, tornou-se cerca de 5.406 vezes mais lenta que o algoritmo de dois ponteiros, de complexidade $O(N+M)$. Os resultados mostram que conforme o tamanho dos vetores aumenta, a diferença de desempenho entre os algoritmos cresce significativamente.